LESSON 19

打个包 struct 一张卡

结构体入门

学完以后，一个同学的编号和分数能装进同一张“卡片”，一起存、一起拿。

编号和分数，分开放容易乱

一个同学有编号，也有分数。用两个数组：`id[i]` 放编号，`score[i]` 放分数，只要 `i` 对齐就行。

可是两样东西分在两处，挪一个忘了挪另一个，编号就对不上分数了。
要是还有年龄、班级，数组就更多了。

更自然的办法：给每个同学做一张卡片，卡片上写着编号和分数。
C++ 里做这种卡片，用 **`struct`**（结构体）。

先画卡片的样子，再做一张

record.cpp

```
 4 struct Record {
 5     int id;
 6     int score;
 7 };
 8 int main() {
 9
10     Record r = {101, 85};
11     cout << r.id << endl;
12     cout << r.score << endl;
13     return 0;
```

第 4 到 7 行画出卡片的样子：叫 Record，上面有 id 和 score 两格。

第 10 行照这个样子做一张卡片 r，按顺序填上 101 和 85。

屏幕上

```
101
85
```

四个要点

`struct Record`	新的类型叫 Record，用起来像 int
`int id; int score;`	卡片上的格子，叫成员
`};`	大括号后面有分号，别漏
`r.score`	点号：取 r 这张卡上的 score 格

struct 写在 main 前面，main 里就能用 Record 做卡片。
r.score 用起来和普通变量一样：能读入、能输出、能计算。

小测：r.score 是几？

```
Record r = {101, 85};
```

1 101

2 85

3 186

4 101 85

答对了！大括号里按成员的顺序填：先 id 是 101，再 score 是 85。

读入：一格一格往里填

read.cpp

```
10      Record r;
11      cin >> r.id >> r.score;
12      cout << r.id << " " << r.score << endl;
```

输入 101 85

```
101 85
```

先做一张空卡片 r，再把读到的数填进它的两格。

编号是 1 到 999，分数是 0 到 100，都是整数，用空格隔开。

猜一猜：改了 score，id 会变吗？

change.cpp

```
10      Record r = {101, 85};
11      r.score = 90;
12      cout << r.id << endl;
13      cout << r.score << endl;
```

屏幕上

```
101
90
```

id 还是 101。卡片上的每一格各管各的，改哪格就只变哪格。

动手改一改

从 read 开始。只动分数那一格。

每人加 5 分

读入编号（1 到 999）和分数（0 到 95）。分数加 5，编号不变，再输出编号和分数，中间空一格。

输入 101 85

```
101 90
```

输入 1 0

```
1 5
```

两张卡片：比分数，报编号

higher.cpp

```
10      Record a, b;
11      cin >> a.id >> a.score;
12      cin >> b.id >> b.score;
13      if (a.score > b.score) {
14          cout << a.id << endl;
15      } else if (b.score > a.score) {
16          cout << b.id << endl;
17      } else {
18          cout << "Tie" << endl;
19      }
```

if 里比的是 .score，输出的是 .id。一张卡片上两样都在，拿哪格用哪格。

输入 101 85 102 90

102

小测：第 14 行输出写成 a.score 会怎样？

输入 101 95 102 90

1 输出 101

2 输出 90

3 输出 95

4 报错

答对了！比较没错，可是输出成了分数，题目要的是编号。

一叠卡片：结构体数组

top.cpp

```
16      int best = 0;
17      for (int i = 1; i < n; i++) {
18          if (s[i].score > s[best].score) {
19              best = i;
20          }
21      }
22      cout << s[best].id << endl;
```

前面先用 Record s[100]; 准备一叠卡片，再用 cin >> s[i].id >> s[i].score; 读入 n 张。

s[i] 是下标为 i 的卡片，s[i].score 是它的分数。

best 记的是**那张卡的下标**，最后输出那张卡的编号。

3 张：101 85、102 92、103 70

102

小测：读完 3 张卡片，s[1].score 是几？

```
101 85 / 102 92 / 103 70
```

1. 85
2. 92
3. 102
4. 70

答对了！下标从 0 开始，s[1] 是第二张，分数是 92。

动手改一改

从 top 开始，比赛规则反过来。

最低分是谁?

先读入 n（1 到 100），再读入 n 行，每行一个编号（1 到 999）和一个分数（0 到 100）。输出最低分的编号，一样低时输出先出现的。

101 85、102 92、103 70

103

1 70、2 70、3 80

1

读报错：大括号后面少了分号

record.cpp（只阅读）

```
4 struct Record {
5     int id;
6     int score;
7 }
```

struct 的大括号和 if、for 的不一样，后面要跟一个分号。

```
main.cpp:7:2: error: expected ';' after struct
```

在第 7 行的 } 后面加上分号就好。

找错：哪一行有问题？

higher.cpp（只阅读）

```
1       if (a.id > b.id) {
2           cout << a.id << endl;
3       } else {
4           cout << b.id << endl;
5       }
```

点代码里的一行

这段想输出分数高的编号。

输入 101 85 102 90，输出 102，对了。

输入 101 90 102 85，还是 102，错了。

点你认为有问题的那一行。

自己挑战

谁高于平均分

先读入 n（1 到 100），再读入 n 行，每行一个编号（1 到 999）和分数（0 到 100）。按输入顺序输出高于平均分的编号，每行一个；一个也没有就输出 None。

输入

```
4
101 60
102 90
103 70
104 100
```

输出（平均 80）

```
102
104
```

先自己写，卡住了再翻下一页。

卡住了？一次看一条提示

提示 1　要读完才知道平均分，所以先把卡片都存进 `Record s[100]`。

提示 2　第二遍再一张张比。平均分可能带小数，想想第 9 课，或者比 `score × n` 和总分。

提示 3　想说 `None`，要知道一个也没有，第 9 课的 `count` 能帮忙。

设计一张自己的卡片

自己画一种卡片的样子，读入一张，再把它漂漂亮亮地输出。主题你自己定。

先做到

写一个 `struct`，至少有两个成员，读入后用点号输出。

选你喜欢的

宠物卡、游戏角色卡、图书卡、球员卡，都可以。名字可以用 `string`。

还想更厉害

读入好几张，找出某一项最厉害的那张。

没有标准答案。写之前说清想要的结果，写完试试最小和最大的输入。

和同桌换着玩

1

换着运行

给同桌的程序填一张卡，看看它怎么展示

2

说一个喜欢的

告诉同桌，你最喜欢这张卡哪里

3

提一个想法

问问同桌：卡片上还可以加哪一格？

听完再改一版。想一想、做出来、试一试、分享、再改，程序就是这样越做越好的。

说得清吗?

Q1 `struct Record` 和 `Record r` 有什么不同?

前者画出卡片的样子，后者照样子做出一张卡片 `r`。

Q2 `r.score = 90` 会不会改到 `r.id`?

不会，每一格各管各的。

Q3 `s[1].score` 是什么?

第二张卡片上的分数，下标从 `0` 开始。

速查卡

```
struct Record { int id; int score; };
```

`{101, 85}`	做卡片时按成员的顺序填
`r.score`	点号取一格，用法像普通变量
`cin >> r.id`	一格一格读入
`Record s[100];`	一叠卡片，s[i].score 是下标 i 那张的分数
`};`	struct 的大括号后面有分号

这些后面再学

递归

函数自己调用自己

排序

把一叠卡片按分数排好

把卡片交给函数

函数的参数也可以是 `Record`

今天学会结构体，几样相关的东西装进一张卡片，一起存、一起拿。

谢谢！

喜欢这节课？欢迎来 GitHub 和我交流。

 cpp-for-youth

下一课：递归入门